

Proving What Happened: An Assurance Case Study in Energy Management

Energy systems are usually judged by whether they keep running. I wanted to ask a harder question: when the connection to central management drops, does local control actually keep making decisions — and could I prove what happened afterwards, to a reviewer who wasn't in the room?

So I built a small laboratory around OpenEMS, an open-source energy management system, and broke it on purpose.

The setup, and the break

The lab models a commercial site: simulated solar generation, a battery, a grid meter, an OpenEMS Edge doing local control, and a central Backend with an InfluxDB time-series store behind it. Everything was pinned — the exact upstream commit, the Java version, SHA-256 hashes of every built artefact — because an experiment you cannot reproduce is an anecdote.

The interesting design decision was how to fail. Killing the Backend process would have collapsed several failure modes into one. Instead I put a small TCP relay between Edge and Backend and stopped only the relay. The Edge stayed up, the Backend stayed up, the database stayed up; only the communication path between them died. That let me test the thing I actually cared about — communication loss — rather than a server crash.

During the outage I changed the operating conditions twice. First I pushed consumption above generation: the battery responded with 4,500 W of discharge. Then I reversed it, generation above consumption: the battery swung to 2,500 W of charge. Grid power held at its 0 W target throughout. This mattered more to me than the process simply surviving. A controller holding a stale set-point looks identical to a working one until the world changes. Here the world changed twice, mid-outage, and the controller followed.

The criterion that refused to pass

I had defined six success criteria before running anything, with a strict rule: all six supported, or the overall claim is not supported. Five passed — measurements stayed available, control continued, nothing transitioned unexpectedly, the interruption was observable, the evidence stayed time-orderable.

The sixth asked whether recovery was fully understandable, including whether telemetry generated during the outage was later backfilled into the central store. My monitoring showed the reconnection clearly, but the channel that should have recorded a successful historical resend stayed null for the whole observation window.

I will admit the temptation: the reconnection was right there in the evidence, and it would have been easy to argue that reconnection implies reconciliation. I published the result as INCONCLUSIVE instead, overall result included. A restored WebSocket proves a socket; it does not prove that history arrived.

Reading the source instead of guessing

The inconclusive result nagged at me, so in the second phase of the project I went looking for the cause — not by rerunning the experiment, but by reading the upstream code.

The answer was in the resend worker. On reconnection, OpenEMS deliberately does not resend history immediately. It schedules the resend for five minutes plus a random delay of up to one hour — a sensible design, since a fleet of devices reconnecting after a regional outage would otherwise hammer the Backend simultaneously. My observation window after reconnection was 48.8 minutes. The scheduler's worst case is 65. There was roughly a one-in-four chance that the resend was simply still pending when I froze the evidence, and everything else I observed is consistent with exactly that.

So the finding changed character: not "data was lost", and not "the feature failed", but "my observation window was shorter than the scheduler's worst-case delay". I kept SC-06 inconclusive — a root cause explains missing evidence, it doesn't substitute for it — and wrote a retest protocol that waits the full 66 minutes. Small confession: I had also enabled the local history store only thirteen minutes before the final run, which limits what gap-detection could see. Both facts are in the repository, because an assurance case that hides its own weaknesses is marketing.

Building the boring layer, carefully

Around the experiment sits the part that looks less exciting and took nearly as much discipline: the governance, risk, compliance and audit structure.

Every claim in the repository lives in a machine-readable model, and the chain — service, dependency, risk, obligation, control, test, evidence, finding — is linked by identifiers, with automated tests that fail the build if any reference dangles. The obligations register maps to NIS2 articles, the UK NIS Regulations and the NCSC Cyber Assessment Framework at provision level, against explicitly fictional entity profiles. Where a fact needed to determine applicability doesn't exist — which member state, what entity size — the record says CONDITIONAL and lists the missing facts, because a mapping is a reason to care about a requirement, not proof of meeting it.

The internal audit program separates three questions that get conflated constantly: is the control well designed, is it actually implemented here, and has it operated effectively over time? Most of my controls honestly conclude "design adequate, operating effectiveness not tested" — one experiment does not make a control effective, and pretending otherwise would undermine the one thing this project is about. The residual-risk register follows the same rule: no risk score was reduced without evidence, so several risks sit at UNASSESSED, visibly.

One more honesty mechanism: every review in the project is labelled SELF_REVIEW. I am one person; relabelling myself as "reviewer" would not create independence, so the limitation is recorded instead of costumed.

What the lab cannot say

Everything ran on one host. That means the experiment can isolate network paths and processes, but a host failure would take the service, its management plane and the local evidence buffer down together — so "local evidence survives" is a claim against connectivity loss only, and the repository says so. The assets are simulated; nothing here proves production resilience, IAM effectiveness, or physical security, and no legal compliance is claimed anywhere. These limits are written into the models, not buried in a footnote.

What the project trained

The method underneath is simple to state and hard to practise: define the service, fail it deliberately, and prove — with evidence that survives scrutiny — that you understood what happened. The specific muscle this project trained is the one I want to use professionally: holding the technical detail and the assurance judgment in the same hand, and refusing to let a green dashboard say more than the evidence does.

The open item is the retest: rerun the outage, wait out the scheduler, and watch whether history actually arrives. If it does, the last criterion flips to supported and the case closes. If it doesn't, that will be a finding worth having — either way, the repository is built so the conclusion has to follow the evidence.

The full repository — models, evidence, audit program and source — is public: github.com/Berkeley/energy-resilience-assurance-lab